

Part 1: Your Hypotheses

Before running any experiments, predict the rejection rate for each defense configuration against each attack type. Write a percentage (0% or 100%) in each cell.

Experiment	Immediate Replay	Delayed Replay	Modified Replay
No defense	0%	0%	0%
Timestamp only	0%	100%	0%
Counter only	100%	100%	0%
All defenses	100%	100%	100%

Part 2: Experiment Results

Experiment 1: No Defense (Baseline)

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	5	0	0%
Delayed replay	5	0	0%
Modified replay	5	0	0%

Experiment 2: Timestamp Only

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	5	0	0%
Delayed replay	0	5	100%
Modified replay	5	0	0%

Experiment 3: Counter Only

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	0	5	100%
Delayed replay	0	5	100%
Modified replay	5	0	0%

Experiment 4: All Defenses

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	0	5	100%
Delayed replay	0	5	100%
Modified replay	0	5	100%

Part 3: Summary Table

Compile all your results into a single table:

Defense	Immediate Replay	Delayed Replay	Modified Replay
No defense	0%	0%	0%
Timestamp only	0%	100%	0%
Counter only	100%	100%	0%
All defenses	100%	100%	100%

Part 4: Screenshots

Terminal output (experiment summary):

Experiment 1: No Defense (Baseline)

```
=====
EXPERIMENT SUMMARY
=====
Defense      Attack      Accepted    Rejected    Rate
-----
none         immediate    5           0           0%
none         delayed      5           0           0%
none         modified     5           0           0%

[SAVED] Results saved to experiment_results.json
christinekh@christinekhs-MacBook-Pro Hydroficient-project %
```

Experiment 2: Timestamp Only

```
=====
EXPERIMENT SUMMARY
=====
Defense      Attack      Accepted    Rejected    Rate
-----
timestamp    immediate    5           0           0%
timestamp    delayed      0           5           100%
timestamp    modified     5           0           0%

[SAVED] Results saved to experiment_results.json
christinekh@christinekhs-MacBook-Pro Hydroficient-project %
```

Experiment 3: Counter Only

```
=====
EXPERIMENT SUMMARY
=====
```

Defense	Attack	Accepted	Rejected	Rate
counter	immediate	0	5	100%
counter	delayed	0	5	100%
counter	modified	5	0	0%

```
[SAVED] Results saved to experiment_results.json
christinekh@christinekhs-MacBook-Pro Hydroficient-project %
```

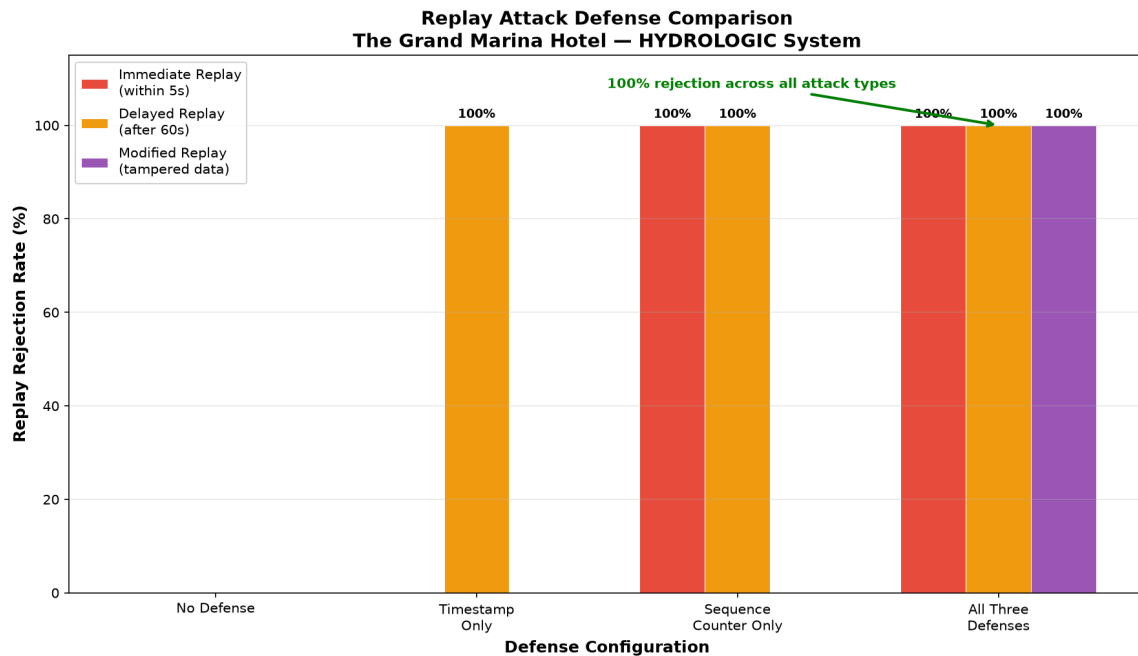
Experiment 4: All Defenses

```
=====
EXPERIMENT SUMMARY
=====
```

Defense	Attack	Accepted	Rejected	Rate
all	immediate	0	5	100%
all	delayed	0	5	100%
all	modified	0	5	100%

```
[SAVED] Results saved to experiment_results.json
christinekh@christinekhs-MacBook-Pro Hydroficient-project %
```

defense_comparison.png chart:



Part 5: Hypothesis Comparison

1. Which predictions were correct?

All of my predictions were correct. The no-defense configuration rejected 0% of all attacks. Timestamp validation rejected 100% of delayed replays but did not stop immediate or modified replays. The sequence counter rejected 100% of immediate and delayed replays but did not stop modified replays. Using all three defenses together rejected 100% of all tested attacks.

2. Which results surprised you? Were there any attack/defense combinations you expected to block that didn't, or vice versa?

The most interesting result was that the sequence counter rejected delayed replays just as effectively as immediate replays. This showed that the counter does not depend on how much time has passed. I also found it useful to see that timestamp validation could not stop an immediate replay because the message was still within the 30-second allowed time window.

3. Why did individual defenses have gaps? Think about what each defense checks and what it doesn't.

Each defense protects against a different weakness. Timestamp validation checks whether a message is recent, but it cannot tell whether a recent message has already been used. The sequence counter detects reused sequence numbers, but it can be bypassed if an attacker changes the sequence number. HMAC detects message tampering, but by itself it does not prevent an attacker from replaying an unchanged, valid message. Combining all three provides layered protection against the different attack types.